Universidad de Buenos Aires
Facultad de Ingeniería
Año 2026 – 1.er Cuatrimestre

Redes de Comunicaciones

Trabajo Práctico n.° 1

Curso 1

Lautaro García Vitale	110028	lagarciav@fi.uba.ar
Dante Gabriel Mele Ientile	109494	dmele@fi.uba.ar

Resumen

En el presente trabajo práctico se analiza el funcionamiento de gRPC como mecanismo de comunicación distribuida sobre la arquitectura TCP/IP, integrando aspectos conceptuales y resultados experimentales obtenidos mediante Wireshark. Para ello se implementó un servicio cliente-servidor en Python a partir de una definición Protocol Buffers y se estudiaron los mensajes intercambiados entre ambas partes.

Las experiencias se desarrollaron en múltiples escenarios: comunicación local mediante la interfaz de *loopback*, comunicación entre dos estaciones dentro de una misma red LAN y ensayos con carga concurrente utilizando múltiples solicitudes paralelas. Asimismo, se evaluó el comportamiento del sistema frente a retardos fijos, retardos variables (*jitter*) y errores simulados del servicio.

A partir de las capturas y estadísticas obtenidas se examinan el establecimiento y cierre de conexiones TCP, el uso de HTTP/2 y sus *streams*, la reutilización de *sockets*, el impacto del RTT en los tiempos de respuesta, el orden relativo de consultas y respuestas concurrentes, y los códigos de estado reportados por gRPC ante condiciones normales y anómalas.

Los resultados muestran cómo gRPC aprovecha la multiplexación provista por HTTP/2 y la serialización eficiente de Protocol Buffers para sostener comunicaciones concurrentes y robustas, incluso ante variaciones de retardo y fallas inducidas.

En esta entrega, además del presente documento, se adjuntan los *scritps* utilizados y las capturas obtenidas en *Wireshark*. Se recomienda filtrar estas últimas mediante `tcp.port == 50051`

Índice

1. Introducción **3**

2. *Google Remote Procedure Calls* (gRPC) **3**

3. Comunicación gRPC capturada en Wireshark **4**
3.1. Escenario 1: cliente y servidor en la misma estación 4
3.1.1. Versión de IP utilizada 5
3.1.2. Puerto de escucha del servidor 5
3.1.3. Ejecución simultánea de dos servidores 5
3.1.4. Consulta simple: paquetes intercambiados 5
3.1.5. Opciones TCP negociadas 6
3.1.6. Persistencia de la conexión 6
3.1.7. Streams HTTP/2 y legibilidad 6
3.2. Escenario 2: cliente y servidor en dos estaciones de la misma LAN 7
3.2.1. Consulta simple y tiempos medidos 7
3.2.2. Origen de la diferencia temporal 8
3.2.3. Impacto en el RTT y negociación TCP 8
3.2.4. Parámetros observados 9
3.3. Comparación de resultados 9

4. Comunicación gRPC con alteraciones **9**
4.1. Escenario 1: `delay = 0`, `jitter = 0`, `errores = 0`, `N = 100` 9
4.2. Escenario 2: `delay = 100 ms`, `jitter = 0`, `errores = 0`, `N = 100` 10
4.3. Escenario 3: `delay = 100 ms`, `jitter = 100 ms`, `errores = 0`, `N = 100` 11
4.4. Escenario 4: `delay = 100 ms`, `jitter = 100 ms`, `errores = 20 %`, `N = 100` 12

5. Conclusiones **14**

1. Introducción

Las aplicaciones modernas suelen requerir mecanismos de comunicación eficientes, confiables y escalables para intercambiar información entre procesos distribuidos. En ese contexto, gRPC surge como una alternativa ampliamente adoptada para la implementación de servicios remotos de alto rendimiento, al combinar el transporte provisto por TCP, las capacidades de multiplexación de HTTP/2 y la serialización compacta de Protocol Buffers.

El objetivo de este trabajo es estudiar, desde una perspectiva práctica, cómo se materializan dichos mecanismos en una implementación real. Para ello se desarrollaron experiencias controladas con cliente y servidor gRPC en Python, complementadas con capturas de tráfico mediante Wireshark. El análisis incluye tanto el comportamiento básico de una consulta RPC como situaciones más exigentes, con múltiples solicitudes concurrentes, retardos artificiales y respuestas con error.

A través de estos ensayos se busca relacionar los conceptos teóricos de las capas de transporte y aplicación con evidencia experimental, permitiendo comprender de qué manera los parámetros de red y las decisiones de diseño del protocolo influyen sobre el desempeño observado.

2. *Google Remote Procedure Calls* (gRPC)

Google Remote Procedure Calls (**gRPC**) es un marco de trabajo de código abierto orientado a la implementación de llamadas a procedimientos remotos (RPC), diseñado para facilitar la comunicación eficiente entre aplicaciones distribuidas. Permite que un cliente invoque métodos ubicados en un servidor remoto como si fueran funciones locales, abstrayendo gran parte de la complejidad asociada al intercambio de mensajes en red.

Además, gRPC se apoya en **HTTP/2** como protocolo de aplicación subyacente, lo que le permite aprovechar características modernas como la multiplexación de múltiples flujos de datos sobre una única conexión, compresión de cabeceras y comunicación bidireccional persistente. Dentro de HTTP/2, cada intercambio se organiza mediante un ***stream***, entendido como un flujo independiente de datos dentro de una conexión multiplexada.

En la capa de transporte, la comunicación se establece típicamente sobre **TCP** (*Transmission Control Protocol*), el cual proporciona un canal confiable, orientado a conexión y de tipo *full-duplex*, garantizando la entrega ordenada de los segmentos transmitidos.

Uno de los elementos centrales de gRPC es el ***channel***, que representa una conexión lógica entre cliente y servidor. Sobre un mismo *channel* pueden coexistir múltiples invocaciones RPC reutilizando la misma conexión física, mejorando la eficiencia y reduciendo la sobrecarga de establecimiento de sesiones.

Para la definición de interfaces y estructuras de datos, gRPC utiliza ***Protocol Buffers***, un mecanismo de serialización binaria compacto y eficiente. Los servicios, métodos y mensajes se describen en archivos con extensión `.proto`, a partir de los cuales se genera automáticamente código cliente y servidor en diversos lenguajes de programación.

Asimismo, gRPC soporta distintos modelos de interacción. Entre ellos se encuentra la ***Unary* RPC**, en la cual el cliente envía una única solicitud y recibe una única respuesta.

También se dispone de ***Bidirectional Streaming* RPC**, modalidad en la que cliente y servidor pueden enviarse múltiples mensajes de forma simultánea e independiente, sin necesidad de respetar un orden estricto entre emisiones.

Cada invocación RPC finaliza con un ***StatusCode***, código que informa si la operación fue exitosa o si ocurrió algún tipo de error (por ejemplo, recurso inexistente, *timeout* o permisos insuficientes). Además, gRPC permite definir un ***deadline***, es decir, un límite máximo de tiempo para completar una llamada remota. Si dicho tiempo se excede, la operación se cancela automáticamente, evitando bloqueos indefinidos y mejorando la robustez del sistema distribuido.

3. Comunicación gRPC capturada en Wireshark

Con el objetivo de estudiar el funcionamiento de gRPC sobre la pila de protocolos TCP/IP, se realizaron capturas con Wireshark en dos escenarios distintos: primero con cliente y servidor ejecutándose en una misma estación, y luego en dos estaciones diferentes dentro de una misma red LAN. A partir de dichas capturas se analizan la interfaz utilizada, la versión de IP empleada, los puertos involucrados, el establecimiento y cierre de conexión TCP, el uso de *streams* HTTP/2 y el impacto temporal de utilizar *loopback* frente a una red física.

3.1. Escenario 1: cliente y servidor en la misma estación

No.	Time	Source	Destination	Protocol	Length	Info
3	14.092605	::1	::1	TCP	88	49935 → 50051 [SYN] Seq=0 Win=65535 Len=0 MSS=16324 WS=64 TSval=44463399 TSecr=0 SACK_PERM
4	14.092712	::1	::1	TCP	88	50051 → 49935 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=16324 WS=64 TSval=448725549 TSecr=44463399 SACK_PERM
5	14.092723	::1	::1	TCP	76	49935 → 50051 [ACK] Seq=1 Ack=1 Win=407808 Len=0 TSval=44463399 TSecr=448725549
6	14.092730	::1	::1	TCP	76	[TCP Window Update] 50051 → 49935 [ACK] Seq=1 Ack=1 Win=407808 Len=0 TSval=448725549 TSecr=44463399
7	14.092914	::1	::1	HTTP2	158	Magic, SETTINGS[0], WINDOW_UPDATE[0]
8	14.092918	::1	::1	HTTP2	122	SETTINGS[0], WINDOW_UPDATE[0]
9	14.092954	::1	::1	TCP	76	50051 → 49935 [ACK] Seq=47 Ack=83 Win=407744 Len=0 TSval=448725549 TSecr=44463399
10	14.092958	::1	::1	TCP	76	49935 → 50051 [ACK] Seq=83 Ack=47 Win=407808 Len=0 TSval=44463399 TSecr=448725549
11	14.093086	::1	::1	HTTP2	85	SETTINGS[0]
12	14.093104	::1	::1	TCP	76	50051 → 49935 [ACK] Seq=47 Ack=92 Win=407744 Len=0 TSval=448725550 TSecr=44463400
13	14.093229	::1	::1	GRPC/PB…	333	HEADERS[1]: POST /Calculator/Sumar, WINDOW_UPDATE[1], DATA[1] (GRPC) (PROTOBUF), WINDOW_UPDATE[0]
14	14.093248	::1	::1	TCP	76	50051 → 49935 [ACK] Seq=47 Ack=349 Win=407488 Len=0 TSval=448725550 TSecr=44463400
15	14.094645	::1	::1	HTTP2	115	SETTINGS[0], WINDOW_UPDATE[0], PING[0]
16	14.094847	::1	::1	TCP	76	49935 → 50051 [ACK] Seq=349 Ack=86 Win=407808 Len=0 TSval=44463401 TSecr=448725551
17	14.095045	::1	::1	HTTP2	93	PING[0]
18	14.095067	::1	::1	TCP	76	50051 → 49935 [ACK] Seq=86 Ack=366 Win=407488 Len=0 TSval=448725552 TSecr=44463401
19	14.095276	::1	::1	GRPC/PB…	218	HEADERS[1]: 200 OK, DATA[1] (GRPC) (PROTOBUF), HEADERS[1]
20	14.095293	::1	::1	TCP	76	49935 → 50051 [ACK] Seq=366 Ack=228 Win=407680 Len=0 TSval=44463402 TSecr=448725552
21	14.095483	::1	::1	HTTP2	93	PING[0]
22	14.095496	::1	::1	TCP	76	50051 → 49935 [ACK] Seq=228 Ack=383 Win=407488 Len=0 TSval=448725552 TSecr=44463402
23	14.095684	::1	::1	HTTP2	93	PING[0]
24	14.095702	::1	::1	TCP	76	49935 → 50051 [ACK] Seq=383 Ack=245 Win=407680 Len=0 TSval=44463402 TSecr=448725552
25	14.095939	::1	::1	TCP	76	49935 → 50051 [FIN, ACK] Seq=383 Ack=245 Win=407680 Len=0 TSval=44463402 TSecr=448725552
26	14.095964	::1	::1	TCP	76	50051 → 49935 [ACK] Seq=245 Ack=384 Win=407488 Len=0 TSval=448725552 TSecr=44463402
27	14.096175	::1	::1	TCP	76	50051 → 49935 [FIN, ACK] Seq=245 Ack=384 Win=407488 Len=0 TSval=448725553 TSecr=44463402
28	14.096202	::1	::1	TCP	76	49935 → 50051 [ACK] Seq=384 Ack=246 Win=407680 Len=0 TSval=44463403 TSecr=448725553

Figura 1: Captura Wireshark de la comunicación gRPC en una misma estación.

Cuando cliente y servidor se ejecutan en la misma computadora, la comunicación aparece en la interfaz `lo0`, correspondiente al dispositivo de *loopback* del sistema operativo. Esto se debe a que los paquetes nunca abandonan la máquina, sino que son encaminados internamente por la pila de red. Por esa razón, no intervienen interfaces físicas como Ethernet o Wi-Fi.

Si se desea forzar el uso de una interfaz física determinada, basta con que el cliente se conecte a la dirección IP real de la placa de red correspondiente en lugar de utilizar `localhost`. Por ejemplo, usando la IP local de Wi-Fi o Ethernet, el sistema operativo enruta el tráfico por dicha interfaz.

3.1.1. Versión de IP utilizada

En la captura de la Figura 1 se observa que las direcciones origen y destino corresponden a `::1`, es decir, la dirección de *loopback* de IPv6. Por lo tanto, la comunicación se realizó mediante IPv6.

Es posible forzar IPv4 utilizando `127.0.0.1`, mientras que utilizando `::1` o resolviendo `localhost` con preferencia IPv6 se empleará IPv6. La validación experimental surge directamente de las direcciones mostradas por Wireshark.

3.1.2. Puerto de escucha del servidor

El primer segmento TCP muestra una conexión desde un puerto efímero del cliente hacia el puerto `50051`, por lo que se concluye que el servidor escucha en dicho puerto. Este puerto no corresponde a una asignación oficial estándar de IANA para gRPC, pero es ampliamente utilizado por convención en ejemplos y aplicaciones de prueba.

3.1.3. Ejecución simultánea de dos servidores

Si el servidor ya se encuentra ejecutándose y se intenta lanzar una segunda instancia usando el mismo puerto, el sistema operativo rechaza la operación con un error del tipo `Failed to add port to server: No address added out of total 1 resolved for '[::]:50051'`. En ese caso no se establece una nueva sesión TCP válida, por lo que no aparece tráfico gRPC adicional significativo en la captura.

3.1.4. Consulta simple: paquetes intercambiados

Durante una única consulta entre cliente y servidor se distinguen tres etapas principales:

- **Establecimiento de conexión TCP (3-way handshake):** segmentos `[SYN]`, `[SYN,ACK]` y `[ACK]`, observados en los paquetes 3, 4 y 5.
- **Intercambio de datos de aplicación:** luego del establecimiento se intercambian tramas correspondientes a HTTP/2 y gRPC, entre ellas mensajes `SETTINGS`, `WINDOW_UPDATE`, `HEADERS`, `DATA` y `PING`. La invocación RPC se observa como:

  `POST /Calculator/Sumar`

  seguida por la respuesta del servidor:

  `200 OK`
- **Cierre ordenado de conexión TCP:** segmentos `[FIN,ACK]`, `[ACK]`, `[FIN,ACK]` y `[ACK]`, correspondientes a los paquetes 25, 26, 27 y 28.

Tomando como inicio el primer segmento del handshake (paquete 3) y como fin el último `ACK` del cierre (paquete 28), se contabilizan 26 tramas capturadas en total. La mayoría de ellas cuentan con `Len=0`, lo que implica que no hay datos útiles enviados, por ejemplo, tratándose de segmentos de confirmación.

El tiempo total de la sesión resulta:

$$14.096202 \text{ s} - 14.092605 \text{ s} = 0.003597 \text{ s} = 3.597 \text{ ms}$$

3.1.5. Opciones TCP negociadas

Durante el establecimiento de conexión se observan las siguientes opciones TCP negociadas:

- `MSS = 16324`
- `Window Scale = 64`
- `SACK_PERM`
- `TSval` y `TSecr`

siendo `SACK_PERM` la opción que habilita la retransmisión únicamente de los segmentos perdidos. Además, `TSval` (*value*) y `TSecr` (*echo reply*) son TCP *timestamps* que permiten un cálculo preciso del RTT en el entorno local.

El tamaño inicial de ventana anunciado por el cliente en el segmento SYN fue:

`Win = 65535  bytes`

Luego de aplicar el factor de escalado negociado, la ventana efectiva pasa a ser mayor, como se observa en los paquetes siguientes.

3.1.6. Persistencia de la conexión

En esta ejecución la conexión no permaneció abierta luego de finalizar la consulta RPC, sino que fue cerrada inmediatamente después de enviada la respuesta. Por lo tanto, no se utilizó persistencia para múltiples invocaciones sucesivas.

El primer segmento de cierre es el paquete 25:

$49935 \rightarrow 50051$ `[FIN,ACK]`

por lo que el cierre fue iniciado por el cliente.

3.1.7. Streams HTTP/2 y legibilidad

En la captura se distinguen dos identificadores de stream HTTP/2:

- **Stream 0:** reservado para control de conexión. Allí aparecen mensajes `SETTINGS`, `PING` y `WINDOW_UPDATE`.
- **Stream 1:** utilizado para la llamada RPC principal, visible en mensajes `HEADERS[1]` y `DATA[1]`.

Parte de la información de aplicación es legible en Wireshark, por ejemplo:

- `POST /Calculator/Sumar`
- `200 OK`

Sin embargo, la carga útil transportada por gRPC aparece serializada como:

`DATA (GRPC) (PROTOBUF)`

por lo que no se visualiza directamente como texto plano.

A diferencia de HTTP/1.1, donde suelen observarse líneas explícitas en texto plano (ASCII) como:

```
GET /sumar?a=10&b=5 HTTP/1.1
POST /api/calculator/sumar HTTP/1.1
```

gRPC utiliza HTTP/2 con tramas binarias multiplexadas y compresión de cabeceras, enviando *payloads* binarios, reduciendo la legibilidad directa de la captura pero mejorando la eficiencia del transporte.

3.2. Escenario 2: cliente y servidor en dos estaciones de la misma LAN

No.	Time	Source	Destination	Protocol	Length	Info
593	2.559311	10.185.3.122	10.185.21.49	TCP	78	50591 → 50051 [SYN, ECE, CWR] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=585431051 TSecr=0 SACK_PERM
598	2.563748	10.185.21.49	10.185.3.122	TCP	74	50051 → 50591 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1408 WS=256 SACK_PERM TSval=411598581 TSecr=585431051
599	2.563835	10.185.3.122	10.185.21.49	TCP	66	50591 → 50051 [ACK] Seq=1 Ack=1 Win=131264 Len=0 TSval=585431055 TSecr=411598581
600	2.564284	10.185.3.122	10.185.21.49	HTTP2	148	Magic, SETTINGS[0], WINDOW_UPDATE[0]
601	2.569090	10.185.21.49	10.185.3.122	HTTP2	112	SETTINGS[0], WINDOW_UPDATE[0]
603	2.569168	10.185.3.122	10.185.21.49	TCP	66	50591 → 50051 [ACK] Seq=83 Ack=47 Win=131264 Len=0 TSval=585431061 TSecr=411598586
604	2.569327	10.185.3.122	10.185.21.49	HTTP2	75	SETTINGS[0]
605	2.569864	10.185.3.122	10.185.21.49	GRPC/P…	326	HEADERS[1]: POST /Calculator/Sumar, WINDOW_UPDATE[1], DATA[1] (GRPC) (PROTOBUF), WINDOW_UPDATE[0]
606	2.570183	10.185.21.49	10.185.3.122	HTTP2	75	SETTINGS[0]
607	2.570236	10.185.3.122	10.185.21.49	TCP	66	50591 → 50051 [ACK] Seq=352 Ack=56 Win=131264 Len=0 TSval=585431062 TSecr=411598587
608	2.572733	10.185.21.49	10.185.3.122	TCP	66	50051 → 50591 [ACK] Seq=56 Ack=352 Win=65024 Len=0 TSval=411598590 TSecr=585431061
609	2.573838	10.185.21.49	10.185.3.122	HTTP2	83	PING[0]
610	2.573881	10.185.3.122	10.185.21.49	TCP	66	50591 → 50051 [ACK] Seq=352 Ack=73 Win=131264 Len=0 TSval=585431065 TSecr=411598591
611	2.574009	10.185.3.122	10.185.21.49	HTTP2	83	PING[0]
612	2.576220	10.185.21.49	10.185.3.122	GRPC/P…	221	HEADERS[1]: 200 OK, DATA[1] (GRPC) (PROTOBUF), HEADERS[1], WINDOW_UPDATE[0]
613	2.576245	10.185.3.122	10.185.21.49	TCP	66	50591 → 50051 [ACK] Seq=369 Ack=228 Win=131136 Len=0 TSval=585431068 TSecr=411598593
614	2.576477	10.185.3.122	10.185.21.49	HTTP2	83	PING[0]
615	2.576837	10.185.3.122	10.185.21.49	TCP	66	50591 → 50051 [FIN, ACK] Seq=386 Ack=228 Win=131136 Len=0 TSval=585431068 TSecr=411598593
616	2.580690	10.185.21.49	10.185.3.122	TCP	66	50051 → 50591 [ACK] Seq=228 Ack=387 Win=65024 Len=0 TSval=411598598 TSecr=585431066
618	2.582924	10.185.21.49	10.185.3.122	HTTP2	83	PING[0]
619	2.582925	10.185.21.49	10.185.3.122	TCP	66	50051 → 50591 [FIN, ACK] Seq=245 Ack=387 Win=65024 Len=0 TSval=411598598 TSecr=585431066
620	2.583019	10.185.3.122	10.185.21.49	TCP	54	50591 → 50051 [RST] Seq=387 Win=0 Len=0

Figura 2: Captura Wireshark de una consulta gRPC entre dos estaciones distintas en la LAN.

En este segundo escenario, cliente y servidor se ejecutan en equipos distintos conectados a la misma red local. Las direcciones IP observadas son `10.185.3.122` y `10.185.21.49`, por lo que el tráfico ya no circula por la interfaz de *loopback*, sino a través de una interfaz física Ethernet o Wi-Fi.

3.2.1. Consulta simple y tiempos medidos

Se repite la misma prueba de una consulta gRPC simple. En la captura se distinguen nuevamente tres etapas:

- **Establecimiento de conexión TCP (3-way handshake):** segmentos `[SYN]`, `[SYN,ACK]` y `[ACK]`, correspondientes a los paquetes 593, 598 y 599.
- **Intercambio de datos:** tramas HTTP/2 y gRPC con mensajes `SETTINGS`, `WINDOW_UPDATE`, `HEADERS`, `DATA` y `PING`. La solicitud aparece como:

  `POST /Calculator/Sumar`

  y la respuesta del servidor como:

  `200 OK`
- **Cierre de conexión:** el cliente inicia el cierre con un segmento `[FIN,ACK]` (paquete 615), el servidor responde con `[ACK]` (paquete 616), luego transmite `[FIN,ACK]` (paquete 619) y finalmente el cliente responde con un segmento `[RST]` (paquete 620), en lugar del `ACK` final habitual.

Tomando como referencia el primer segmento del establecimiento y el último paquete observado, el tiempo total de la sesión es:

$$2.583019 \text{ s} - 2.559311 \text{ s} = 0.023708 \text{ s} = 23.708 \text{ ms}$$

Comparado con el escenario local:

$$23.708 \text{ ms} > 3.597 \text{ ms}$$

se evidencia un incremento significativo de latencia.

3.2.2. Origen de la diferencia temporal

En la interfaz de *loopback* sólo intervienen:

- procesamiento del sistema operativo,
- copias en memoria,
- cambios de contexto entre procesos,
- ejecución local de cliente y servidor.

En una red LAN real se agregan además:

- acceso al medio físico,
- transmisión por cable o Wi-Fi,
- retardo de propagación,
- procesamiento en *switches* o puntos de acceso,
- colas de espera y *buffering*,
- posibles retransmisiones de capa física.

Por ello, la latencia total resulta mayor.

3.2.3. Impacto en el RTT y negociación TCP

En el escenario local, el RTT es extremadamente pequeño, permitiendo una negociación TCP casi instantánea. En la LAN el RTT aumenta debido al trayecto real entre ambos hosts, lo cual repercute en:

- temporizadores iniciales TCP,
- cálculo del RTO,
- crecimiento inicial de la ventana de congestión,
- tiempo de establecimiento de sesión.

3.2.4. Parámetros observados

Durante el establecimiento de conexión se observan las siguientes opciones TCP:

- `MSS = 1460`
- `Window Scale = 64` (cliente)
- `Window Scale = 256` (servidor)
- `SACK Permitted`
- `TCP Timestamps`

El valor de MSS es consistente con una MTU Ethernet típica de 1500 bytes, a diferencia del valor considerablemente mayor observado en la interfaz de *loopback*.

3.3. Comparación de resultados

La comunicación gRPC sobre una misma estación utiliza la interfaz de *loopback* y presenta tiempos mínimos debido a la ausencia de tránsito físico. En cambio, al ejecutar cliente y servidor en dos equipos distintos dentro de una misma LAN, aparecen retardos propios de la red y aumenta el RTT, impactando tanto en el tiempo total de consulta como en el comportamiento dinámico de TCP.

Asimismo, en ambos escenarios se verifica que gRPC opera sobre TCP y HTTP/2, utilizando *streams* multiplexados y cargas serializadas mediante Protocol Buffers.

4. Comunicación gRPC con alteraciones

En esta sección utilizaremos los *scripts* `server_delay.py` y `client_parallel.py` provistos por la cátedra para simular un *delay* medio y *jitter* entre consultas y respuestas y errores de servicio en el primero; `N` peticiones gRPC en el segundo.

4.1. Escenario 1: `delay = 0`, `jitter = 0`, `errores = 0`, `N = 100`

HTTP/2 utiliza la siguiente convención para asignar *stream* ID:

- `[0]`: Reservado para controles de conexión (tal y como se indicó en la sección anterior);
- `[impares]`: Reservado para el cliente;
- `[pares]`: Reservado para el servidor.

Es por esto que se observa que, luego del *stream* 0 y dado que el servidor no envía datos, los *streams* IDs de esta comunicación serán 1, 3, 5, ..., 199.

Esta convención permite la multiplexación, posibilitando que el cliente envíe `N` peticiones simultaneas en una misma conexión TCP con un stream ID impar —el mismo que utilizará el servidor para responder—.

El RTT medio obtenido en esta comunicación es 34.613 ms y su desvío estándar resultó 12.824 ms.

En la Figura 3 se observan las primeras 26 tramas de esta comunicación. Nótese que en la trama n.° 549 el cliente envía la primera solicitud de suma "`HEADERS[1]: POST / Calculator ...`" y en la 563 el servidor responde "`HEADERS[1]: 200 OK, DATA[1] ...`". Además, nótese que en la trama 551, se envían 23 *streams* en conjunto (`ID [3], ... ID [49]`).

Este comportamiento permite observar la característica concurrente de las transacciones, en donde el cliente no espera a recibir el "`200 OK`" del servidor para enviar los paquetes.

No.	Time	Source	Destination	Protocol	Length	Info
538	6.330596400	::1	::1	TCP	76	49335 → 50051 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
539	6.330737700	::1	::1	TCP	76	50051 → 49335 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
540	6.330822600	::1	::1	TCP	64	49335 → 50051 [ACK] Seq=1 Ack=1 Win=65280 Len=0
541	6.331549500	::1	::1	HTTP2	146	Magic, SETTINGS[0], WINDOW_UPDATE[0]
542	6.331623400	::1	::1	TCP	64	50051 → 49335 [ACK] Seq=1 Ack=83 Win=65280 Len=0
543	6.346757600	::1	::1	HTTP2	110	SETTINGS[0], WINDOW_UPDATE[0]
544	6.346818300	::1	::1	TCP	64	49335 → 50051 [ACK] Seq=83 Ack=47 Win=65280 Len=0
545	6.347035200	::1	::1	HTTP2	73	SETTINGS[0]
546	6.347088100	::1	::1	TCP	64	50051 → 49335 [ACK] Seq=47 Ack=92 Win=65280 Len=0
547	6.347118400	::1	::1	HTTP2	73	SETTINGS[0]
548	6.347168300	::1	::1	TCP	64	49335 → 50051 [ACK] Seq=92 Ack=56 Win=65280 Len=0
549	6.347905800	::1	::1	GRPC/PB(<UNKNOWN>)	327	HEADERS[1]: POST /Calculator/Sumar, WINDOW_UPDATE[1], DATA[1] (GRPC) (PROTOBUF), WINDOW_UPDATE[0]
550	6.347959700	::1	::1	TCP	64	50051 → 49335 [ACK] Seq=56 Ack=355 Win=65024 Len=0
551	6.348273900	::1	::1	GRPC/PB(<UNKNOWN>)	1273	HEADERS[3]: POST /Calculator/Sumar, WINDOW_UPDATE[3], DATA[3] (GRPC) (PROTOBUF), HEADERS[5]: POST /Calculator/Sumar
552	6.348343000	::1	::1	TCP	64	50051 → 49335 [ACK] Seq=56 Ack=1564 Win=63744 Len=0
553	6.348398900	::1	::1	HTTP2	81	PING[0]
554	6.348458400	::1	::1	TCP	64	49335 → 50051 [ACK] Seq=1564 Ack=73 Win=65280 Len=0
555	6.348517000	::1	::1	GRPC/PB(<UNKNOWN>)	327	HEADERS[51]: POST /Calculator/Sumar, WINDOW_UPDATE[51], DATA[51] (GRPC) (PROTOBUF), HEADERS[53]: POST /Calculator/S
556	6.348550600	::1	::1	TCP	64	50051 → 49335 [ACK] Seq=73 Ack=1827 Win=63488 Len=0
557	6.348658900	::1	::1	HTTP2	81	PING[0]
558	6.348687500	::1	::1	TCP	64	50051 → 49335 [ACK] Seq=73 Ack=1844 Win=63488 Len=0
559	6.348995200	::1	::1	GRPC/PB(<UNKNOWN>)	127	HEADERS[61]: POST /Calculator/Sumar, WINDOW_UPDATE[61], DATA[61] (GRPC) (PROTOBUF), WINDOW_UPDATE[0]
560	6.349031300	::1	::1	TCP	64	50051 → 49335 [ACK] Seq=73 Ack=1907 Win=63488 Len=0
561	6.349840100	::1	::1	GRPC/PB(<UNKNOWN>)	127	HEADERS[63]: POST /Calculator/Sumar, WINDOW_UPDATE[63], DATA[63] (GRPC) (PROTOBUF), WINDOW_UPDATE[0]
562	6.349887000	::1	::1	TCP	64	50051 → 49335 [ACK] Seq=73 Ack=1970 Win=63488 Len=0
563	6.350484300	::1	::1	GRPC/PB(<UNKNOWN>)	220	HEADERS[1]: 200 OK, DATA[1] (GRPC) (PROTOBUF), HEADERS[1], WINDOW_UPDATE[0]

Figura 3: Captura Wireshark escenario 1.

4.2. Escenario 2: `delay = 100 ms`, `jitter = 0`, `errores = 0`, `N = 100`

En este escenario la comunicación ocurre en un único *socket*, es decir, en una única comunicación TCP, esto se puede verificar observando que existe un único *3-way handshake* al inicio de la conexión tal y como se observa en la Figura 4.

Nuevamente, al igual que en el escenario anterior, se observa el patrón concurrente de las transacciones gracias a multiplexación y a la identificación mediante el *stream* ID. En la Figura 5 se observa que las respuestas del servidor tienden a estar ordenadas pero no de forma excluyente —véase tramas 2236 (`HEADERS [85]`), 2238 (`HEADERS [89]`) y 2240 (`HEADERS [87]`)—.

No.	Time	Source	Destination	Protocol	Length	Info
1954	19.955559200	::1	::1	TCP	76	58203 → 50051 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
1955	19.955798700	::1	::1	TCP	76	50051 → 58203 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM
1956	19.955884000	::1	::1	TCP	64	58203 → 50051 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1957	19.956899200	::1	::1	HTTP2	146	Magic, SETTINGS[0], WINDOW_UPDATE[0]
1958	19.956913800	::1	::1	HTTP2	110	SETTINGS[0], WINDOW_UPDATE[0]
1959	19.956977100	::1	::1	TCP	64	58203 → 50051 [ACK] Seq=83 Ack=47 Win=65280 Len=0
1960	19.957007000	::1	::1	TCP	64	50051 → 58203 [ACK] Seq=47 Ack=83 Win=65280 Len=0
1961	19.957272100	::1	::1	HTTP2	73	SETTINGS[0]
1962	19.957558600	::1	::1	HTTP2	73	SETTINGS[0]
1963	19.957626000	::1	::1	TCP	64	58203 → 50051 [ACK] Seq=92 Ack=56 Win=65280 Len=0
1964	19.957985400	::1	::1	GRPC/PB(<UNKNOWN>)	327	HEADERS[1]: POST /Calculator/Sumar, WINDOW_UPDATE[1], DATA[1] (GRPC) (PROTOBUF), WINDOW_UPDATE[0]
1965	19.958248800	::1	::1	GRPC/PB(<UNKNOWN>)	674	HEADERS[3]: POST /Calculator/Sumar, WINDOW_UPDATE[3], DATA[3] (GRPC) (PROTOBUF), HEADERS[5]: POST /Calculator/Sumar
1966	19.958633900	::1	::1	GRPC/PB(<UNKNOWN>)	127	HEADERS[27]: POST /Calculator/Sumar, WINDOW_UPDATE[27], DATA[27] (GRPC) (PROTOBUF), WINDOW_UPDATE[0]
1967	19.959385400	::1	::1	GRPC/PB(<UNKNOWN>)	127	HEADERS[29]: POST /Calculator/Sumar, WINDOW_UPDATE[29], DATA[29] (GRPC) (PROTOBUF), WINDOW_UPDATE[0]

Figura 4: Captura Wireshark escenario 2.

2236	20.107568100	::1	::1	GRPC/PB(<UNKNOWN>)	118	HEADERS[85]: 200 OK, DATA[85] (GRPC) (PROTOBUF), HEADERS[85]
2237	20.107632800	::1	::1	TCP	64	58203 → 50051 [ACK] Seq=6492 Ack=2531 Win=62976 Len=0
2238	20.109613100	::1	::1	GRPC/PB(<UNKNOWN>)	118	HEADERS[89]: 200 OK, DATA[89] (GRPC) (PROTOBUF), HEADERS[89]
2239	20.109669300	::1	::1	TCP	64	58203 → 50051 [ACK] Seq=6492 Ack=2585 Win=62720 Len=0
2240	20.109845800	::1	::1	GRPC/PB(<UNKNOWN>)	118	HEADERS[87]: 200 OK, DATA[87] (GRPC) (PROTOBUF), HEADERS[87]
2241	20.109963900	::1	::1	TCP	64	58203 → 50051 [ACK] Seq=6492 Ack=2639 Win=62720 Len=0

Figura 5: Captura Wireshark escenario 2 (continuación).

El RTT medio obtenido en esta comunicación es 124.453 ms y su desvío estándar resultó 3.428 ms. El parámetro `jitter = 0` en el *script* asegura que la red no aumente su variabilidad. Sin embargo, el desvío estándar de unos pocos milisegundos existe por retrasos

propios del Sistema Operativo. Nótese que en este escenario, la media del RTT resultó aproximadamente 100 ms mayor que en el escenario anterior.

Para observar los 100 ms en una sola transacción, podemos considerar el primer *stream* ID:

```
No.: 1964  Time: 19.957985400  Info: HEADERS[1]: POST /Calculator/Sumar (...)
No.: 2138  Time: 20.061814900  Info: HEADERS[1]: 200 OK (...)
```

Computando la diferencia de tiempo entre la solicitud del cliente y la respuesta del servidor, obtenemos un *delay* aproximado de 103 ms.

Al extrapolar el concepto de *delay* a la Internet global, este no se limita solo a la velocidad de transmisión, sino que se manifiesta en cualquier punto donde los datos deban viajar, procesarse o esperar en cola. Percibimos este retraso en múltiples contextos reales de la red, por ejemplo: cuando los paquetes de datos deben recorrer miles de kilómetros de fibra óptica (*propagation delay*), cuando un servidor web remoto debe buscar información pesada en una base de datos (*processing delay*), o cuando la infraestructura de red está saturada por alto tráfico y los paquetes deben esperar su turno en las colas de los *routers* (*queuing delay*), causando lentitud generalizada.

4.3. Escenario 3: `delay = 100 ms`, `jitter = 100 ms`, `errores = 0`, `N = 100`

El RTT medio obtenido en este escenario es 125.358 ms y su desvío estándar 60.740 ms. El primer valor es correcto y consistente con los escenarios anteriores, en cambio, el segundo, es muy alto (dado por el `jitter = 100`) lo que significa que existen paquetes que tardaron unos 65 ms y otros más de 180 ms.

En esta situación se observa de forma muy marcada en el desorden en las respuestas del servidor (Figura 6). Podemos resumir el funcionamiento de esta manera:

- El cliente envía las solicitudes con un *stream* ID impar de forma secuencial (1, 3, 5, 7, ...)
- El servidor recibe las peticiones pero, por culpa del *jitter* introducido, cada respuesta debe esperar un tiempo aleatorio.
- La multiplexación permite que el cliente pueda seguir enviando solicitudes sin bloquearse por las demoras del servidor y, a su vez, que el servidor no deba esperar, por ejemplo, que la respuesta No. 5 esté lista para enviar la No. 42.
- A pesar del desorden temporal en las respuestas del servidor, el cliente puede identificar qué respuesta corresponde a qué pregunta gracias al *stream* ID.

887	6.875014400	::1	::1	GRPC/PB(<UNKNOWN>)	131 HEADERS[73]: 200 OK, DATA[73] (GRPC) (PROTOBUF), HEADERS[73], WINDOW_UPDATE[0]
888	6.875117600	::1	::1	TCP	64 59108 → 50051 [ACK] Seq=6420 Ack=527 Win=64768 Len=0
889	6.878969700	::1	::1	GRPC/PB(<UNKNOWN>)	118 HEADERS[97]: 200 OK, DATA[97] (GRPC) (PROTOBUF), HEADERS[97]
890	6.879056000	::1	::1	TCP	64 59108 → 50051 [ACK] Seq=6420 Ack=581 Win=64768 Len=0
891	6.880012300	::1	::1	GRPC/PB(<UNKNOWN>)	118 HEADERS[11]: 200 OK, DATA[11] (GRPC) (PROTOBUF), HEADERS[11]
892	6.880055400	::1	::1	TCP	64 59108 → 50051 [ACK] Seq=6420 Ack=635 Win=64768 Len=0
893	6.880063000	::1	::1	GRPC/PB(<UNKNOWN>)	118 HEADERS[53]: 200 OK, DATA[53] (GRPC) (PROTOBUF), HEADERS[53]
894	6.880753200	::1	::1	TCP	64 59108 → 50051 [ACK] Seq=6420 Ack=689 Win=64768 Len=0
895	6.881085300	::1	::1	GRPC/PB(<UNKNOWN>)	118 HEADERS[23]: 200 OK, DATA[23] (GRPC) (PROTOBUF), HEADERS[23]
896	6.881126500	::1	::1	TCP	64 59108 → 50051 [ACK] Seq=6420 Ack=743 Win=64768 Len=0
897	6.881381200	::1	::1	GRPC/PB(<UNKNOWN>)	118 HEADERS[61]: 200 OK, DATA[61] (GRPC) (PROTOBUF), HEADERS[61]

Figura 6: Captura Wireshark escenario 3.

Un efecto análogo en la Internet global, el (*jitter*) puede observar en servicios de *streaming* de video en vivo (Twitch, transmisiones deportivas, etc.). En estos contextos, si los paquetes de datos atraviesan rutas dinámicas o sufren congestiones variables, llegarán al destino con retardos impredecibles y desordenados. Al no disponer de los paquetes en el momento exacto que deben reproducirse, se generan congelamientos en el video o cortes e interrupciones en el audio. Para mitigar este fenómeno, las aplicaciones de *streaming* emplean búferes de *jitter* (*jitter buffers*), los cuales retienen intencionalmente la reproducción unos milisegundos para absorber la variabilidad, dar tiempo a que lleguen los paquetes atrasados y reordenarlos antes de presentarlos al usuario.

4.4. Escenario 4: `delay = 100 ms`, `jitter = 100 ms`, `errores = 20%`, `N = 100`

En este escenario mantenemos los valores del anterior pero agregamos un porcentaje de error. Se obtiene un RTT medio de 116.106 ms y un desvío de 57.593 ms y 26 errores. Es interesante destacar que, como el código `server_delay.py` evalúa la probabilidad de error y luego ejecuta el delay (*Fast-fail*), todas las peticiones con error tienen un RTT mucho menor al del promedio (del orden de los 20 ms).

En la consola, al ejecutar el código, se puede observar que el estado es `status = INTERNAL`, indicando que el mensaje alcanzó el destino pero el servidor encontró una excepción interna al intentar procesar la operación.

Al inspeccionar los paquetes de respuesta enviados por el servidor en *Wireshark*, se puede determinar el resultado analizando el campo `grpc-status` que viaja en los *Trailers* de HTTP/2 (Figuras 7 y 8):

- Los paquetes sin error devuelven el código `grpc-status: 0 (OK)` .
- Los paquetes con error devuelven el código `grpc-status: 13 (INTERNAL)`. Además, se observa que estos paquetes omiten por completo la trama `DATA` y envían un único bloque `HEADERS`

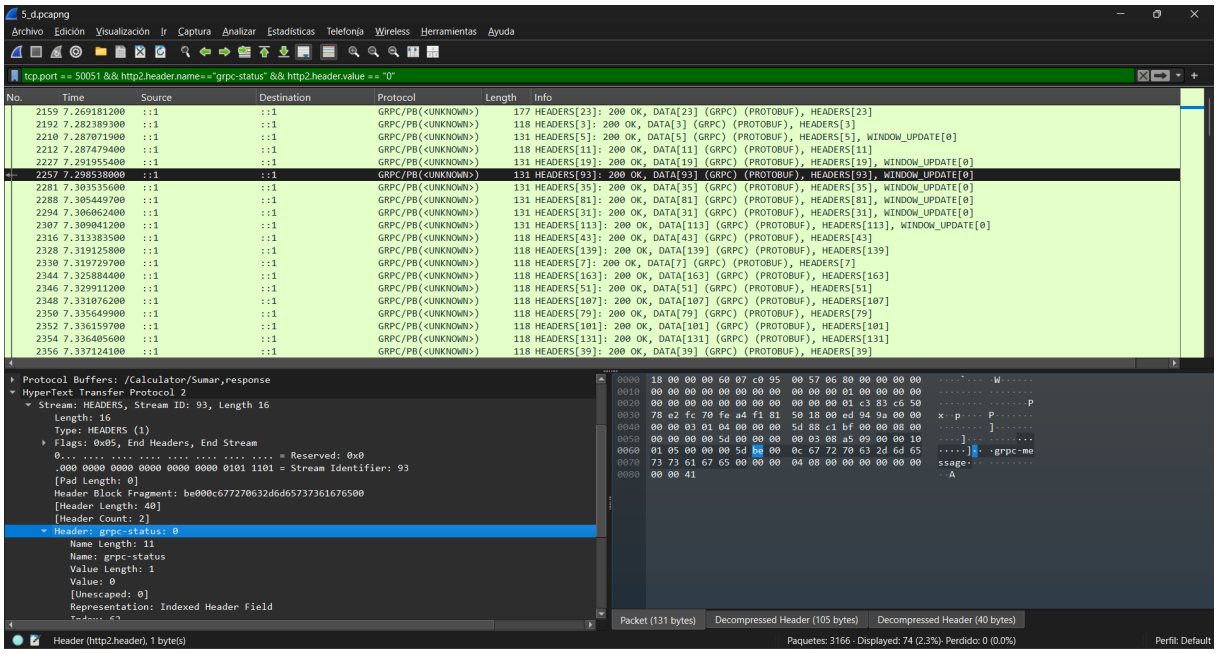

Figura 7: grpc-status: 0.

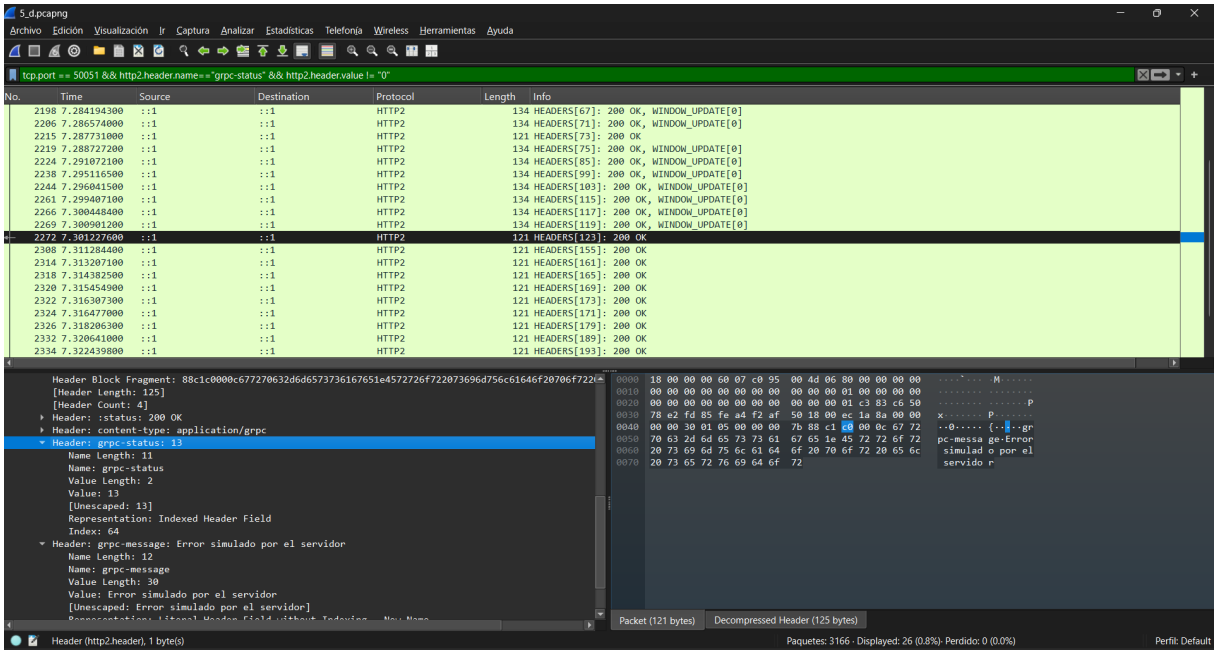

Figura 8: grpc-status: 13.

gRPC contiene en su estándar 16 códigos de error para múltiples situaciones, las más comunes son:

- `DEADLINE_EXCEEDED`: código 4, cuando expira el tiempo máximo de espera.
- `UNAVAILABLE`: código 14, servidor inalcanzable por fallas de red.
- `UNIMPLEMENTED`: código 12, método inexistente.
- `UNAUTHENTICATED`: código 16, credenciales inválidas.

Es importante destacar que un error en el servicio gRPC no implica un error en HTTP/2.

Pues el segundo únicamente tiene la responsabilidad de entregar el paquete con éxito. En consecuencia, se sigue observando un estado HTTP/2 de "`200 OK`" con la diferencia de que aquellos con error no tendrán el frame `DATA` (ya que gRPC la omite). Este comportamiento se puede observar en la Figura 9, en donde el `HEADERS[23]` (exitoso) contiene el frame `DATA` y, en cambio, el `HEADERS[25]` (erróneo) lo omite.

2156	7.267568300	::1	::1	HTTP2	121 HEADERS[25]: 200 OK
2157	7.267625000	::1	::1	TCP	64 50768 → 50051 [ACK] Seq=1347 Ack=315 Win=65024 Len=0
2158	7.268675400	192.168.56.1	224.2.2.2	UDP	62 49665 → 8995 Len=30
2159	7.269181200	::1	::1	GRPC/PB(<UNKNOWN>)	177 HEADERS[23]: 200 OK, DATA[23] (GRPC) (PROTOBUF), HEADERS[23]

Figura 9: Repuesta HTTP/2

5. Conclusiones

A lo largo de este trabajo práctico, se logró contrastar la teoría subyacente a gRPC, HTTP/2 y TCP con el comportamiento empírico observado en la red mediante Wireshark. El análisis del tráfico permitió validar el funcionamiento de la multiplexación de HTTP/2, permitiendo la transmisión concurrente de múltiples peticiones mediante *Stream IDs* independientes sobre un único *socket* TCP.

Asimismo, la simulación de distintos escenarios demostró el impacto real de parámetros como el retardo y la fluctuación (*jitter*). Se evidenció empíricamente que, frente a latencias variables, las respuestas pierden su orden secuencial inicial, un fenómeno que en la Internet global afecta críticamente a servicios en tiempo real.

Finalmente, la inyección de fallos permitió comprender la separación de responsabilidades entre capas: un error lógico de aplicación (gRPC status 13) es manejado de forma independiente al transporte, manteniendo intacto el estado de éxito (`200 OK`) en la capa HTTP/2. En conjunto, estas observaciones demuestran la eficiencia, concurrencia y robustez de gRPC para el desarrollo de sistemas distribuidos modernos.

Referencias

[1] J. F. Kurose y K. W. Ross, *Redes de computadoras: Un enfoque descendente*, 7. ed., Pearson Educación, Madrid, 2017.

[2] gRPC Authors, *gRPC Documentation*, disponible en: `https://grpc.io/docs/`, consultado en abril de 2026.

[3] The Linux Foundation, *Protocol Buffers Documentation*, disponible en: `https://protobuf.dev/`, consultado en abril de 2026.

[4] Internet Engineering Task Force (IETF), *Hypertext Transfer Protocol Version 2 (HTTP/2)*, RFC 9113, junio de 2022. Disponible en: `https://www.rfc-editor.org/rfc/rfc9113.html`, consultado en abril de 2026.

[5] Wireshark Foundation, *Wireshark User's Guide*, disponible en: `https://www.wireshark.org/docs/`, consultado en abril de 2026.

[6] Python Software Foundation, *Python 3 Documentation*, disponible en: `https://docs.python.org/3/`, consultado en abril de 2026.

[7] Internet Assigned Numbers Authority (IANA), *Service Name and Transport Protocol Port Number Registry*, disponible en: `https://www.iana.org/assignments/service-names-port-numbers/`, consultado en abril de 2026.